

Système de gestion d'une pharmacie

Projet de programmation Java

Oussama Tarchi • Ali Zouine
Taha Rajel • Abdessamad Lyamani

Encadrant : Pr. Said El Kafhali | 2025 / 2026

Plan de la présentation

01 Contexte & objectifs

02 Analyse des besoins

03 Conception & architecture

04 Base de données

05 Réalisation technique

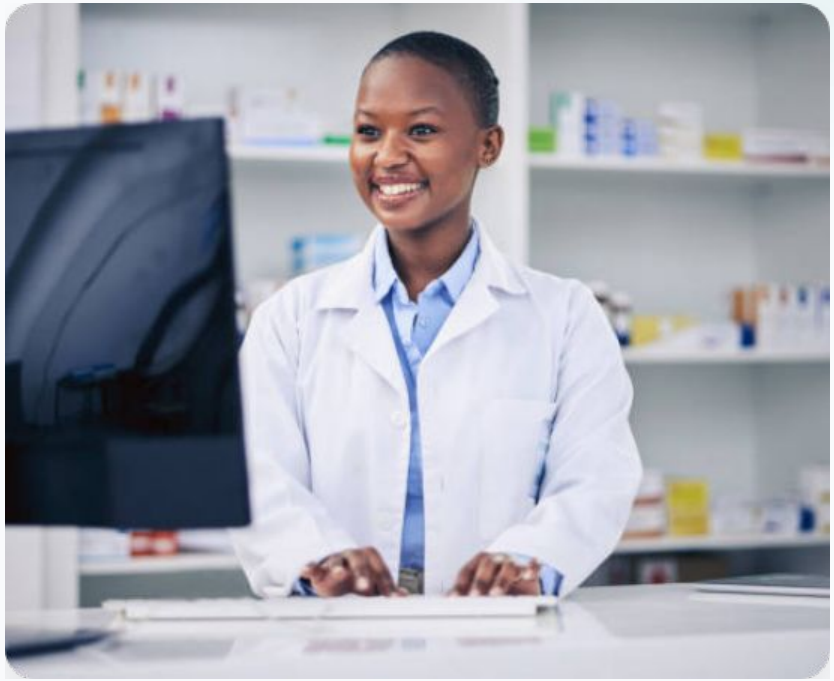
06 Fonctionnement de l'appli

07 Tests & problèmes résolus

08 Limites & perspectives

BESOIN D'UN SYSTÈME INFORMATIQUE

- **Grand volume de données :** Gestion de milliers de références de médicaments.
- **Stock en temps réel :** Éviter les ruptures et gérer les péremptions.
- **Suivi des ventes :** Analyse quotidienne du chiffre d'affaires.
- **Fournisseurs :** Centralisation des commandes et réceptions.
- **Sécurité :** Réduction drastique des erreurs humaines.



LA SOLUTION : NOTRE APPLICATION

Une application Desktop développée en Java ,elle facilite la gestion des médicaments, des clients, des fournisseurs, des ventes et des commandes , et elle stocke les données dans une base mysql locale.

Elle vise à optimiser le travail du pharmacien et à réduire les erreurs de gestion.

Stock Intelligent

Suivi en temps réel avec alertes automatiques de péremption et de rupture.

Ventes & Factures

Panier dynamique et génération instantanée de factures au format .txt.

Traçabilité Médicale

Gestion rigoureuse des ordonnances liées aux fiches clients.

Dashboard d'Analyse

Visualisation du CA et des statistiques de performance par période.

ORGANISATION DU CODE SOURCE



controller
Gère la logique métier et fait le lien entre la vue et les données (JDBC).

dao
Contient les classes d'accès aux données (SQL) pour chaque entité du système.

factures
Dossier interne dédié au stockage des fichiers **.txt** générés après chaque vente.

model
Définit les entités : Client, Medicament, Vente, Ordonnance, etc.

view
Interface utilisateur développée avec **Java Swing** (Filtres, Tableaux, Onglets).

Main
Point d'entrée principal qui initialise et lance l'application ..

Modèle de données — 9 classes

Client

nom
prénom
téléphone
adresse

Medicament

nomCommercial
composition
forme
dosage
…

Fournisseur

nomSociete
contact
adresse

Ordonnance

idClient
medicament
medecin
dateOrdonnance
notes

Vente

idClient
idOrdonnance
dateVente
Liste<LigneVente>

LigneVente

idVente
idmédicament
quantite
prixUnitaire

Commande

dateComande
statut
idFournisseur
Liste<LigneCommande>

LigneCommande

idCommande
idMédicament
quantiteCommande
prixAchat

Pharmacie

nom
adresse
telephone
email
responsable

Base de données — gestion_pharmacie

9 tables MySQL

client

id, nom, prénom, téléphone, adresse

medicament

id, nomCommercial, prix, quantiteStock, datePeremption, seuilAlerte

fournisseur

id, nomSociete, contact, adresse

ordonnance

id, idClient ➜ client, medecin, dateOrdonnance

vente

id, idClient ➜ client, idOrdonnance ➜ ordonnance, dateVente

lignevente

(idVente ➜ vente, idMedicament ➜ medicament), quantite, prixUnitaire

commande

id, idFournisseur ➜ fournisseur, dateCommande, statut

lignecommande

(idCommande ➜ commande, idMedicament ➜ medicament), quantiteCommandee

pharmacie

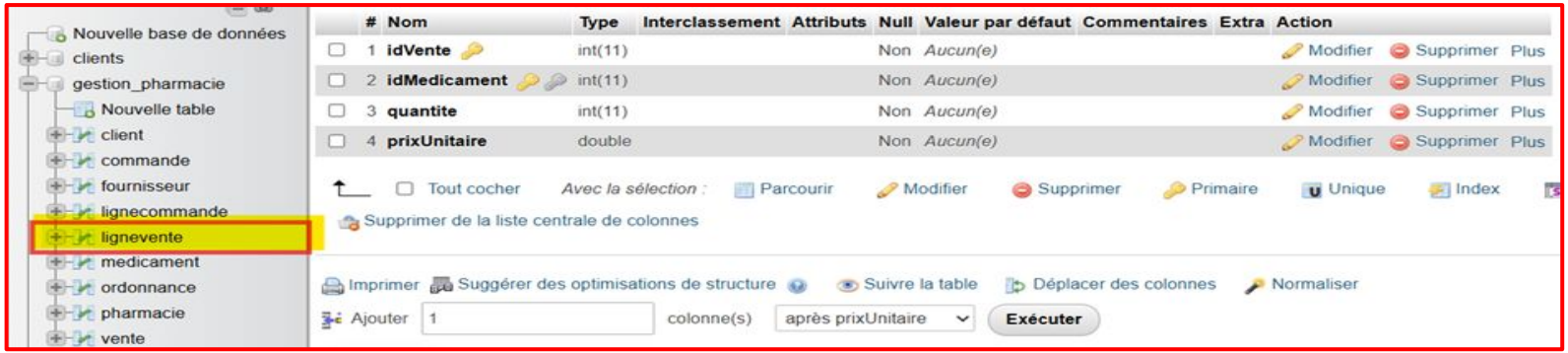
id, nom, adresse, telephone, email, responsable

Base de données — gestion_pharmacie

XAMPP

Intégrité des données :

- **Clé primaire composite** (`idVente` + `idMedicament`).
- **Objectif :** Empêcher la duplication d'un médicament dans une même facture/commande.





Accès aux données : JDBC & couche DAO

Connexion MySQL

- Connecteur mysql-connector-j-9.7.0.jar
- Classe Utilitaire : connexion singleton partagée
- getConnection() réutilise la connexion existante
- closeConnection() libère la ressource proprement

```
conn = DriverManager.getConnection(URL, USER,
PASSWORD);
// PreparedStatement protège
// contre les injections SQL
```

Couche DAO

`ajouter()`	→ INSERT
`modifier()`	→ UPDATE
`supprimer()`	→ DELETE
`getAll()`	→ SELECT *
`getById()`	→ SELECT … WHERE id=?

Accès aux données : JDBC & couche DAO “ Écriture, Sécurité, Lecture et Mapping ”

Écriture & Sécurité

Pour les opérations **C.U.D** (Create, Update, Delete)

- Interface : PreparedStatement
- Méthode : executeUpdate()
- Protection contre les **Injections SQL**

```
public void supprimer(int id) {  1 usage
    String sql = "DELETE FROM Client WHERE id=?";
    try (PreparedStatement ps = Utilitaire.getConnection().prepareStatement(sql)) {
        ps.setInt( parameterIndex: 1, id);
        ps.executeUpdate();
    } catch (SQLException e) {
        System.out.println(" Erreur : " + e.getMessage());
    }
}
```

Lecture & Mapping

Pour les opérations de type **Read** (getAll, getById)

- Méthode : executeQuery()
- Stockage : ResultSet
- Traitement : while(rs.next())

```
public List<Client> getAll() {
    List<Client> list = new ArrayList<>();
    String sql = "SELECT * FROM Client";
    try (Statement st = Utilitaire.getConnection().createStatement();
         ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(new Client(
                rs.getInt( columnLabel: "id"),
                rs.getString( columnLabel: "nom"),
                rs.getString( columnLabel: "prenom"),
                rs.getString( columnLabel: "telephone"),
                rs.getString( columnLabel: "adresse")
```

Couche Contrôleur — logique métier

ClientController
CRUD client + historique des achats

MedicamentController
CRUD médicament, stock, péremption, recherche

PharmacieController
Enregistrement / mise à jour fiche pharmacie

OrdonnanceController
Ajout, vérification client & validité ordonnance

CommandeController
Cycle commande : En attente → Validée → Livrée

StockController
Alertes rupture, péremption, valeur totale stock

FournisseurController
CRUD fournisseurs de médicaments

VenteController
Création vente, validation, facture auto, historique

DashboardController
Stats : ventes/produit, ventes/client, CA/période

Interface graphique — Java Swing

GestionPharmacieView → JTabbedPane → 8 onglets

Pharmacie

Infos de l'établissement

Clients

Ajout / modif. / suppression / recherche

Médicaments

Gestion complète + recherche par nom

Fournisseurs

Gestion des fournisseurs

Ordonnances

Saisie et consultation

Ventes

Panier, validation, historique

Stock

Alertes rupture & péremption, valeur

Dashboard

Stats : produit, client, CA / période



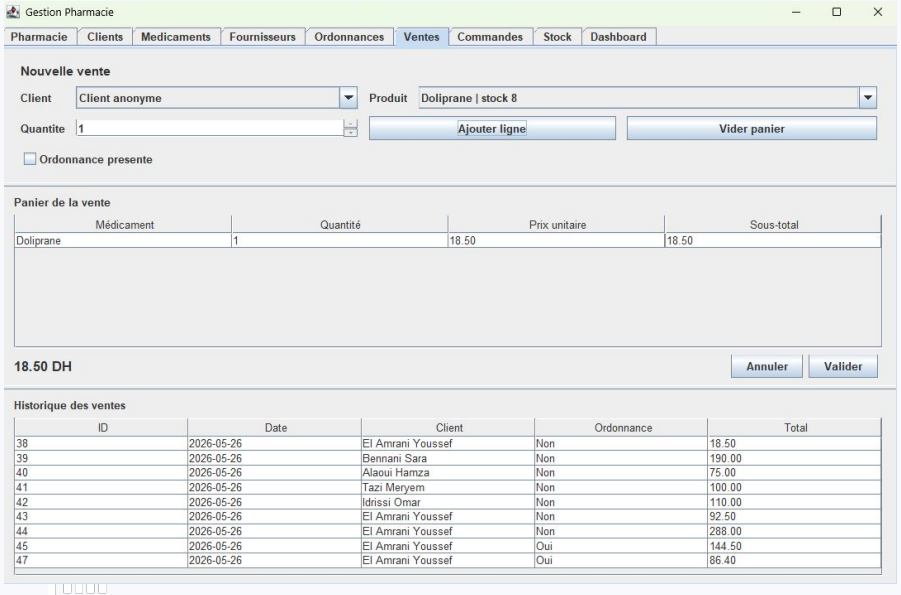
Fonctionnalités clés

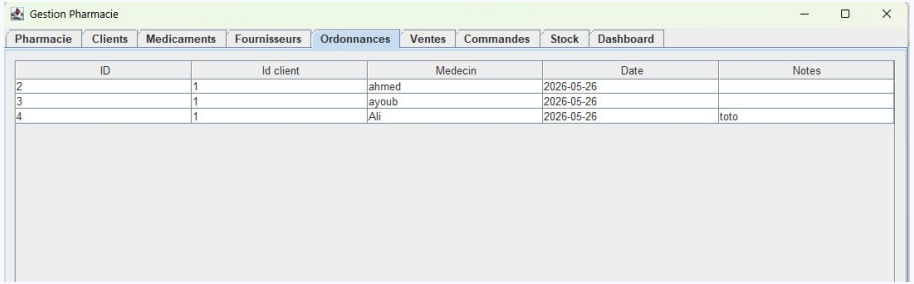
Ventes

Panier en temps réel · avec ou sans ordonnance · validation + MAJ stock automatique

Ordonnances

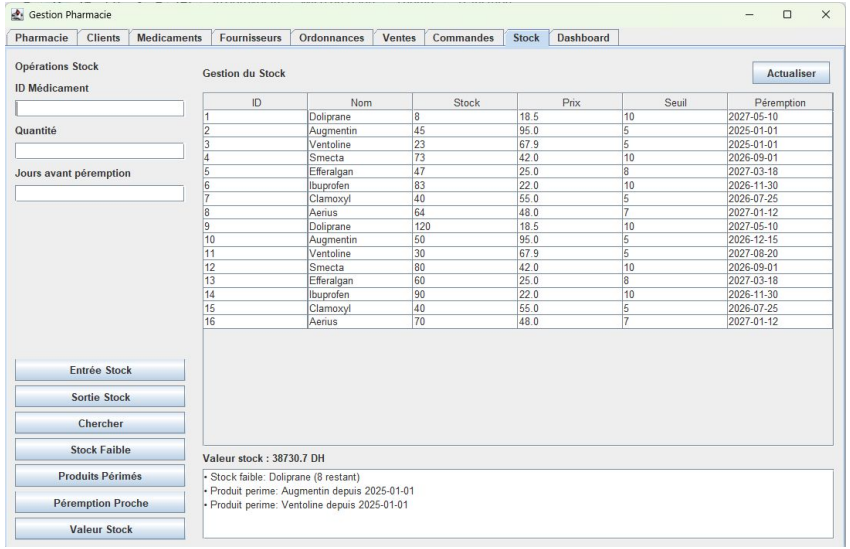
Saisie depuis l'écran Ventes · association au client et à la vente en un clic





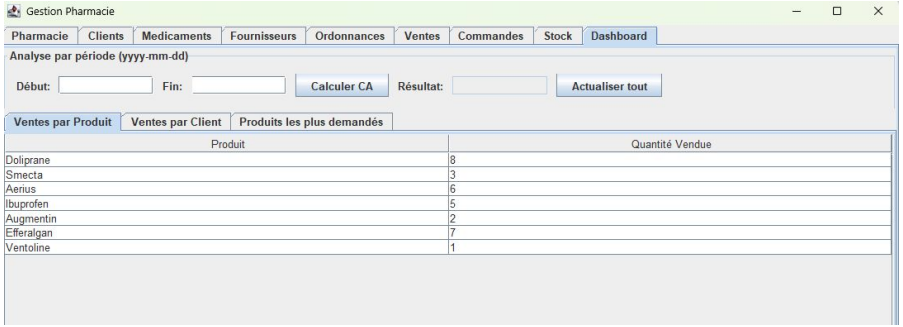
Stock & Alertes

Stock faible, péremption < 30 jours, produits périmés, valeur totale du stock



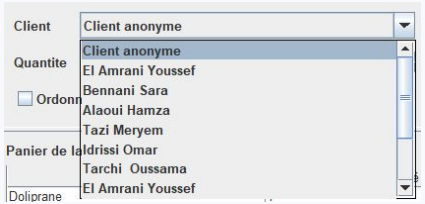
Dashboard

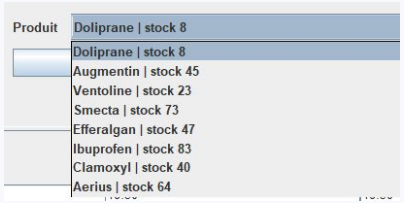
Ventes / produit, ventes / client, CA par période, produits les plus demandés



Processus d'une vente — étape par étape

1	2	3	4	5	6
Sélectionner un client	Choisir un médicament + qté	Ajouter ligne → vente créée en BD	Cocher ordonnance si nécessaire	Valider → vérif. stock	Facture .txt générée auto












```
FACTURE PHARMACIE
──────────────────────────────────
Facture N° : <id>
Client ID  : <idClient>
Date       : yyyy-MM-dd
Ordonnance : N°X ou Sans
──────────────────────────────────
Détails des achats :
- Médicament | Qté | Prix | Sous-total : xx.xx DH
──────────────────────────────────
TOTAL À PAYER : xx DH
```

Problèmes rencontrés

#01 Création de la vente avant l'identifiant

CAUSE

« Ajouter ligne » cliqué sans vente en base → violation de la clé étrangère idVente.

IMPACT

La première ligne échouait systématiquement — aucun médicament ne pouvait être ajouté.

SOLUTION

Le bouton crée désormais la vente au 1er ajout. L'ID est stocké dans idVenteEnCours et réutilisé.

#02 Association tardive de l'ordonnance

CAUSE

L'ordonnance saisie n'était jamais liée à la vente — champ idOrdonnance restait à zéro.

IMPACT

Impossible de vérifier l'appartenance de l'ordonnance au bon client lors de la validation.

SOLUTION

validerVente() appelle enregistrerOrdonnanceSiPresente() qui lie l'ordonnance avant persistance.

Problèmes rencontrés

#03 Double incrémentation du stock

CAUSE

« Réceptionner livraison » et « Changer statut → Livrée » pouvaient tous deux mettre à jour le stock.

IMPACT

Stock doublé (ex. 100→200 au lieu de 150) — incohérence grave dans les données.

SOLUTION

Statut « Livrée » retiré du sélecteur manuel. La réception est l'unique déclencheur — état final immuable.

#04 Séparation DAO / Controller mal définie

CAUSE

Logique métier et requêtes SQL coexistaient dans les mêmes classes.

IMPACT

Débogage difficile, répartition du travail en équipe compliquée, évolution risquée.

SOLUTION

Séparation stricte : les DAO gèrent la BDD, les Controllers gèrent la logique et les actions.

Problèmes rencontrés

#05

Code dupliqué dans les panels Swing

CAUSE

Lecture de champs, validation et affichage d'erreurs réécrits dans chaque panel.

IMPACT

Corrections manuelles partout — risque d'incohérences entre modules.

SOLUTION

Classe utilitaire ViewUtil avec méthodes partagées : lireInt(), lireDouble(), afficherErreur()...

✓5 problèmes identifiés · 5 solutions mises en œuvre · Architecture, qualité et robustesse renforcées

Tests réalisés

Clients

Ajout • Modification • Suppression

✓ Données cohérentes en BDD

Médicaments

Ajout • Recherche par nom

✓ Affichage immédiat

Ventes

Création multi-lignes • Ordonnance • Génération facture

✓ Stock décrémenté, facture.txt créée

Commandes

Création • Changement statut • Réception livraison

✓ Stock incrémenté à la réception

Résultat général : toutes les fonctionnalités principales validées avec succès.

Limites et perspectives

Bien que les objectifs initiaux soient remplis, plusieurs axes d'amélioration permettront de renforcer la robustesse et l'aspect professionnel de la solution.

Visualisation de données

Intégrer des **histogrammes et des courbes** via JFreeChart pour une lecture intuitive du dashboard, remplaçant les simples tableaux textuels pour le stock et les ventes.

Facturation professionnelle

Permettre l'**exportation automatique en format PDF** à l'aide d'iText, facilitant l'archivage et l'impression des factures pour les clients et la comptabilité.

Sécurité & Accès

Ajouter un **système d'authentification robuste** avec gestion de mots de passe pour restreindre l'accès aux données sensibles du stock et des ventes.

Analyses Prédictives

Proposer des **analyses fines du chiffre d'affaires** mensuel et des tendances de consommation par catégorie de médicament pour optimiser le réapprovisionnement.

Conclusion

- Application complète couvrant tous les besoins du cahier des charges
- Architecture en 4 couches : Vue · Contrôleur · DAO · Modèle
- Difficultés surmontées : ordonnances, lignes de vente .
- Compétences consolidées : Java · Swing · JDBC · MySQL
- Base solide, extensible avec PDF, graphiques et authentification

Merci pour votre attention

Oussama Tarchi · Ali Zouine · Taha Rajel · Abdessamad Lyamani